



ITI · INTRODUCTION TO PYTHON TRACK

Python Programming for AI Applications

Lab 3 — Files, Packages & Custom Exceptions

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Lab 3 of 6
Focus	Reading/writing files, building your own package, and raising custom exceptions — the same patterns behind every dataset loader and preprocessing utility

Before You Start

This lab builds directly on today's lecture: you'll read and write files the way a dataset loader does, package a set of related functions the way scikit-learn or numpy are packaged internally, and raise custom exceptions the way a real ML pipeline protects itself from bad input.

Lab Roadmap

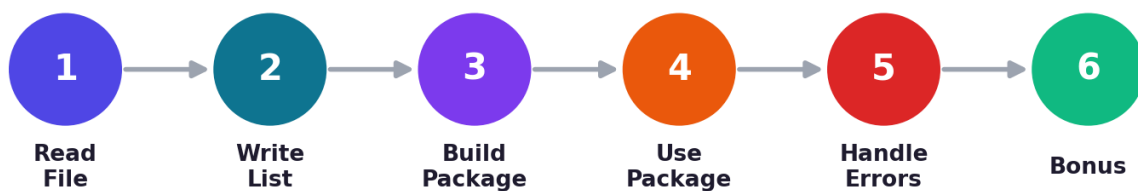


Figure L.1 — Six steps: read a file, write a list, build a package, use it, handle errors, then the bonus round.

✓ QUICK CHECK — Before You Start

Q1. What does the with statement give you when opening a file?

Answer: Automatic closing of the file, even if an error happens inside the block — no need to call `.close()` yourself.

Q2. What's the difference between a module and a package?

Answer: A module is a single `.py` file; a package is a folder containing multiple related modules, importable as one unit.

Q3. How do you stop a bad input from crashing your whole program?

Answer: Wrap the risky code in a `try/except` block, and optionally raise your own exception with a clear message.

Read a File Line by Line

01

Load Sensor Readings into a List

Write a Python program that reads a file line by line and stores each line into a list — exactly the first step of loading any raw dataset before you can process it.

- Use `open()` with mode `"r"`, or better, the `with` statement.
- Loop over the file object directly (for `line in fp:`) to read one line at a time.
- Each element of your list should be one line of the file — strip the trailing `"\n"` if you want clean strings.

AI connection: This is the exact first step of every custom dataset loader — before `pandas.read_csv()` existed, this loop was how everyone loaded data.

Starter code:

```
readings = []
with open("sensor_log.txt", "r") as fp:
    for line in fp:
        readings.append(line.strip())

print(readings)
```



Write a List to a New File

02

Save a List of Model Names to Disk

Create a list of strings — a handful of model names — then add your own name to it, and write the whole list out to a new file, one entry per line.

- Build the list first: `model_names = ["logistic_regression", "random_forest", "svm"]`.
- Append your name to the list with `.append(your_name)`.
- Open a new file in mode "w" and write each item on its own line (remember `write()` doesn't add `"\n"` automatically).

AI connection: This is the same pattern used to write experiment logs, model registries, or a run's list of feature names out to disk for later reference.

Starter code:

```
model_names = ["logistic_regression", "random_forest", "svm"]
model_names.append("your_name")

with open("model_registry.txt", "w") as fp:
    for name in model_names:
        fp.write(name + "\n")
```

Build a Package

03

Create a metrics_toolkit Package

Create a package directory called `metrics_toolkit` and add a file called `my_functions.py` inside it, containing four functions: `sum`, `subtract`, `divide`, and `multiply`.

- A package needs a folder — `metrics_toolkit/` — with an `__init__.py` file (it can be empty) so Python recognizes it as a package.
- Inside `metrics_toolkit/`, create `my_functions.py` with four plain functions, each taking two numbers.
- This mirrors how `scikit-learn` organizes `sklearn/metrics/`, `sklearn/preprocessing/`, and so on — one folder per responsibility.

AI connection: Any time you build a reusable set of helper functions for feature engineering or evaluation metrics across multiple notebooks, this exact package structure is what makes them importable everywhere.

metrics_toolkit/my_functions.py

```
def sum_values(a, b):  
    return a + b  
  
def subtract_values(a, b):  
    return a - b  
  
def divide_values(a, b):  
    return a / b  
  
def multiply_values(a, b):  
    return a * b
```

Use the Package + Handle Errors**04****Build a Calculator on Top of Your Package**

Use the metrics_toolkit package in a program that takes an operand from the user (0 → sum, 1 → subtract, 2 → divide, 3 → multiply) and two numbers, then applies custom validation rules before calling your functions.

- Import your functions with `from metrics_toolkit.my_functions import sum_values, subtract_values, divide_values, multiply_values`.
- If one of the two numbers is 0 and the operand is subtract, raise a `ValueError` with the message "subtracting zero from Number".
- If one of the two numbers is 0 and the operand is divide, raise a `ZeroDivisionError` with the message "can't divide with zero".
- If one of the two numbers is 0 and the operand is multiply, raise a `ValueError` with the message "Multiply with Zero".

AI connection: This is exactly the input-validation layer every AI pipeline needs before it calls a metrics or preprocessing function — catching bad values (like a feature that's zero when it shouldn't be) before they silently corrupt a result.

Starter code:



```
from metrics_toolkit.my_functions import sum_values, subtract_values, divide_values,
multiply_values

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
operand = int(input("0=sum, 1=subtract, 2=divide, 3=multiply: "))

try:
    if operand == 0:
        print(sum_values(a, b))
    elif operand == 1:
        if a == 0 or b == 0:
            raise ValueError("subtracting zero from Number")
        print(subtract_values(a, b))
    elif operand == 2:
        if a == 0 or b == 0:
            raise ZeroDivisionError("can't divide with zero")
        print(divide_values(a, b))
    elif operand == 3:
        if a == 0 or b == 0:
            raise ValueError("Multiply with Zero")
        print(multiply_values(a, b))
except (ValueError, ZeroDivisionError) as e:
    print(f"Invalid operation: {e}")
```